Source Code ID: D6

Code:

```
def number_check(*args, func=None):
    """Check if arguments are numbers."""
    func = func or inspect.stack()[2][3]
    for var in args:
        if not isinstance(var, numbers.Number):
            name = type(var).__name__
            raise DigitError(f'Function {func} expected number, {name} got
instead.')
```

Grok output:

1. Purpose and responsibility

The main purpose of `number_check` is to validate that every positional argument it receives is an instance of `numbers.Number`.

If any argument fails the check, the function raises a `DigitError` whose message identifies both the expected type and the name of the function that (by default) performed the call.

Its responsibility is therefore purely defensive: it either succeeds silently or aborts with a typed exception.

2. Execution flow

1. The function is entered with a variable number of positional arguments (`*args`) and an optional keyword argument `func` (default `None`).

2. The statement `func = func or inspect.stack()[2][3]` is executed.

   - If the caller supplied a non-`None` value for `func`, that value is kept.

   - Otherwise the name of the function two frames up the call stack is obtained via `inspect.stack()` and assigned to `func`.

3. A `for` loop iterates over each element of `args`.

4. For every element the condition `not isinstance(var, numbers.Number)` is evaluated.

   - When the condition is true, the local name `name` is set to the string name of the argument's type and a `DigitError` is raised with a formatted message that includes both `func` and `name`. Execution of the function terminates.

   - When the condition is false, the loop continues with the next argument.

5. If every argument passes the type test, the loop ends and the function returns normally (implicitly returning `None`).

3. Data and state changes

- **Input parameters**
  - `*args` – zero or more objects that will be type-checked.
  - `func` – optional; when supplied it is expected to be a value usable in an f-string (normally a function name). Defaults to `None`.
- **Return value**
  The function has no explicit `return` statement. When it completes without raising, it yields the implicit `None`.
- **Local variables**
  - `func` – re-bound either to the supplied argument or to the result of `inspect.stack()[2][3]`.
  - `var` – loop variable bound successively to each element of `args`.
  - `name` – created only on the failure path; holds `type(var).__name__`.
- **Object state**
  No mutable objects visible in the fragment are modified. The only observable effect is the possible raising of an exception.

4. Dependencies

- `inspect.stack` – called to obtain the current call stack when `func` is not supplied.
- `numbers.Number` – used as the reference type for the `isinstance` test.
- `DigitError` – the exception class that is instantiated and raised on failure.
- Built-ins: `isinstance`, `type`, `__name__` attribute access, and f-string formatting.

The modules or scopes that define `inspect`, `numbers` and `DigitError` are not shown in the provided source.

5. Error handling and special behaviors

- The sole error-handling action is the explicit `raise DigitError(...)`.

- No `try`/`except` blocks are present; any exception raised by `inspect.stack`, by attribute access, or by the construction of the error message propagates to the caller.

- Dynamic behaviour: when `func` is omitted the function inspects the runtime call stack (`inspect.stack()[2][3]`) to recover a function name. The precise meaning of stack index `[2]` is determined by the call depth at the moment of invocation and cannot be verified from this fragment alone.

6. Assumptions and limitations

- The code assumes that `inspect`, `numbers` and `DigitError` are bound names in the enclosing scope.

- It assumes that `inspect.stack()` returns a sequence whose third element (`[2]`) is itself a sequence whose fourth element (`[3]`) is a usable function name.

- It assumes that every element of `args` can be passed to `isinstance` and that `type(var).__name__` yields a string.

- Behaviour when `args` is empty, when `func` is an arbitrary non-string object, or when the call-stack depth is insufficient cannot be determined from the visible source.

- The function does not return a value on the success path, nor does it modify any external state; any broader contract (for example, how callers are expected to use the raised `DigitError`) lies outside the provided code.

Commented [A1]: Hallucination: Factual Mismatch

Reason: It can be determined from the visible code.

Ground Truth: The loop simply executes zero times, and because there is no explicit return, the function reaches the end and returns None.